Manual técnico de despliegue

Modelo predictivo de tarifas Uber · FastAPI · Docker · Google Cloud Run

1. Propósito y arquitectura

La solución recibe parámetros de un viaje desde una página web HTML/JavaScript. La interfaz envía JSON a POST /predict de FastAPI. El backend calcula variables derivadas, carga el modelo Gradient Boosting y devuelve la tarifa estimada en USD. La aplicación se distribuye en una imagen Docker.

2. Requisitos

Python 3.11, pip, Git, Docker Engine/Desktop, 2 GB de RAM recomendados, acceso a internet para descargar dependencias y cuenta GitHub. Para Cloud Run: proyecto Google Cloud con facturación habilitada, permisos de despliegue y Google Cloud CLI (gcloud).

3. Archivos del repositorio

app/main.py: API; app/static/index.html: frontend; train.py: limpieza, entrenamiento y métricas; model/uber_model.joblib: modelo serializado; model/metrics.json: métricas; tests/test_api.py: pruebas; Dockerfile: imagen; docker-compose.yml: orquestación local; requirements.txt: dependencias.

4. Entrenamiento reproducible

Copiar el CSV original a data/uber.csv. El script filtra valores faltantes, tarifas fuera de 2.5–200 USD, pasajeros fuera de 1–6, coordenadas fuera del área configurada y distancias fuera de 0.1–100 km. Usa una división 80/20 con semilla 42.

```
python -m venv .venv
# Activar entorno y luego:
pip install -r requirements-dev.txt
python train.py --data data/uber.csv
pytest -q
```

5. Ejecución local sin Docker

```
uvicorn app.main:app --host 127.0.0.1 --port 8080
```

Abrir http://localhost:8080 para el frontend, http://localhost:8080/docs para Swagger y http://localhost:8080/health para la comprobación de salud.

6. Construcción y ejecución con Docker

```
docker build -t uber-fare-predictor:1.1 .
docker run --rm -p 8080:8080 uber-fare-predictor:1.1
# Alternativa: docker compose up --build
```

La imagen usa python:3.11-slim, instala dependencias fijadas, copia la aplicación y el modelo, ejecuta un usuario sin privilegios y escucha en el puerto configurable PORT (8080 por defecto). El CSV original no se incorpora a la imagen.

7. Prueba de integración de API

```
curl http://localhost:8080/health
curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" -d "{\"pickup_datetime\":\"2015-05-07T19:52:00Z\",\"pickup_latitude\":40.738354,\"pickup_longitude\":-73.999817,\"dropoff_latitude\":40.723217,\"dropoff_longitude\":-73.999512,\"passenger_count\":1}"
```

El endpoint POST /predict devuelve estimated_fare_usd, distance_km y note. Para entradas inválidas, devuelve HTTP 422. La documentación interactiva /docs permite reproducir la solicitud sin curl.

8. Estrategia PaaS: Google Cloud Run

Cloud Run ejecuta contenedores administrados con escalado automático y HTTPS. Elegir región cercana a los usuarios, establecer límites de recursos y configurar acceso público solo si la aplicación debe ser pública.

```
gcloud auth login
gcloud config set project TU_PROJECT_ID
gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com
gcloud run deploy uber-fare-predictor --source . --region us-central1 --allow-unauthenticated --memory 1Gi
```

El comando --source . solicita construir y desplegar desde el código fuente; requiere permisos adecuados, facturación y servicios habilitados. Registrar la URL HTTPS entregada y verificar /health, /docs y una predicción. El despliegue remoto no se ha ejecutado en esta entrega.

9. Portabilidad, seguridad y mantenimiento

Mantener versiones fijadas, excluir datos originales de la imagen, no almacenar claves en Git, usar identidad de servicio con permisos mínimos, restringir CORS si se separa el frontend, revisar costos de Cloud Run, fijar un límite de instancias y actualizar dependencias con pruebas de regresión. El modelo es ilustrativo y no estima precios vigentes.

Anexo A. Procedimiento actualizado y evidencias locales

Comandos de construcción y ejecución comprobados:

- sudo docker build --no-cache -t uber-fare-predictor:1.1 .
- sudo docker run --rm -p 8081:8080 uber-fare-predictor:1.1
- Abrir http://localhost:8081 y http://localhost:8081/docs

La terminal muestra Successfully built, Successfully tagged y respuestas HTTP 200 para GET /, GET /docs, GET /openapi.json y POST /predict. La petición GET /favicon.ico devuelve 404, sin afectar la API.

Anexo B. Despliegue propuesto en Cloud Run

Requisitos: proyecto de Google Cloud con facturación habilitada, gcloud CLI autenticado, permisos para Cloud Run y Cloud Build, y control de costos. Las alertas presupuestarias no bloquean automáticamente el gasto.

- gcloud auth login
- gcloud config set project ID_DEL_PROYECTO
- gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com
- gcloud run deploy uber-fare-predictor --source . --region us-central1 --allow-unauthenticated --port 8080

Ejecutar estos comandos desde la raíz del repositorio. El despliegue desde fuente usa Cloud Build y requiere permisos adecuados; confirmar en la consola la región, URL HTTPS y acceso público antes de declarar éxito. Validar GET /health, GET /docs y POST /predict sobre la URL entregada. No se logra el despliegue para evitar el cobro.